

Desafio de Programação: Inventário de RPG com `std::variant` (C++17)

Este desafio foi projetado para você praticar conceitos de **Programação Orientada a Objetos (POO)** em C++ moderno, utilizando o polimorfismo estático (seguro em relação a tipos) fornecido por `std::variant` e `std::visit`, em vez do polimorfismo dinâmico tradicional (herança e funções virtuais).



O Cenário

Você está desenvolvendo a lógica de inventário para um jogo de RPG de texto. O jogador pode carregar diferentes tipos de itens em sua mochila. No entanto, em vez de herdar todos os itens de uma classe abstrata `Item`, cada tipo de item deve ser uma classe completamente independente e otimizada para o seu próprio propósito.

Você deve criar um sistema que armazene esses itens heterogêneos em um único contêiner (`std::vector`) e seja capaz de interagir com cada um de forma específica.



Objetivos de Aprendizado

- Criar classes coesas com encapsulamento correto em C++.
- Utilizar `std::variant` como um contêiner de tipo seguro para tipos heterogêneos.
- Aplicar o `std::visit` junto com o padrão auxiliar `overloaded` para despachar ações baseadas no tipo guardado no variant em tempo de execução.
- Evitar o uso de ponteiros brutos, `dynamic_cast` ou herança tradicional.



Requisitos do Sistema

1. As Classes de Itens

Você deve implementar pelo menos três classes de itens distintas:

1. `Arma`
 - Atributos privados: `nome` (string), `dano` (int).
 - Métodos: Construtor, getters para os atributos e um método para obter uma representação textual do ataque.
2. `Pocao`
 - Atributos privados: `nome` (string), `cura` (int), `tipo` (string - ex: "Vida" ou "Mana").
 - Métodos: Construtor e getters.
3. `Pergaminho`
 - Atributos privados: `nome` (string), `feitico` (string), `custoMana` (int).
 - Métodos: Construtor e getters.

2. O Tipo Variant

Defina um apelido de tipo (alias) utilizando `using` para representar qualquer item possível:

```
using Item = std::variant<Arma, Pocao, Pergaminho>;
```

3. A Classe Inventario

Esta classe gerenciará a coleção de itens do jogador.

- Atributos privados: Um `std::vector<Item>` para armazenar os itens.
- Métodos públicos:
 - `void adicionarItem(const Item& item) :` Adiciona um novo item ao vetor.
 - `void listarItens() const :` Percorre o vetor e exibe o nome e o tipo de cada item.
 - `void usarTodosOsItens() :` Percorre o vetor e executa a ação correspondente para cada tipo de item usando `std::visit` e um visitante sobrecarregado (`overloaded`).



O Padrão `overloaded` (Dica de C++17)

Para usar o `std::visit` com funções lambda de forma limpa, você precisará declarar a seguinte estrutura auxiliar no topo do seu código (antes da classe `Inventario`):

```
template<class... Ts>
struct overloaded : Ts... { using Ts::operator()...; };

template<class... Ts>
overloaded(Ts...) -> overloaded<Ts...>;
```



Saída Esperada no Console

Ao instanciar os itens no método `main`, adicioná-los ao inventário e chamar `usarTodosOsItens()`, a saída no terminal deve ser semelhante a esta:

```
--- USANDO ITENS DO INVENTÁRIO ---
[Arma] Você empunha a 'Espada de Aço' e ataca causando 45 de dano físico!
[Poção] Você bebe a 'Poção de HP' e restaura 50 de Vida!
[Pergaminho] Você lê o 'Pergaminho de Bola de Fogo' e conjura o feitiço gastand
[Arma] Você dispara com o 'Arco Élfico' e ataca causando 35 de dano físico!
-----
```



Guia de Implementação (Passo a Passo)

1. **Configuração Inicial:** Crie um arquivo chamado `meu_inventario.cpp` e inclua as bibliotecas necessárias: `<iostream>`, `<vector>`, `<variant>`, `<string>` e `<utility>`.
2. **Classes de Itens:** Escreva as classes `Arma`, `Pocao` e `Pergaminho` garantindo que os atributos sejam privados e que existam construtores adequados.
3. **Aliás do Variant:** Crie o `using Item = ...` e declare o padrão `overloaded`.
4. **Classe Inventário:** Desenvolva a classe `Inventario`. Na função `usarTodosOsItens()`, monte a estrutura do `std::visit` com lambdas capturando por referência ou por valor constante (`const auto&`).
5. **Função Main:** - Instancie um objeto `Inventario`.

- Adicione pelo menos uma instância de cada classe criada.
- Chame as funções de listagem e de uso dos itens.

6. **Compilação:** Lembre-se de compilar indicando a versão correta do C++ (C++17 ou superior):

```
g++ -std=c++17 meu_inventario.cpp -o meu_inventario  
./meu_inventario
```



Boa sorte! Escrever o código do zero vai solidificar muito o seu entendimento de gerenciamento de memória e design de software moderno em C++.